Code-Layout, Readability and Reusability

Date	4 July 2026
Team ID	
Project Name	Comprehensive measure of well-being
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

This document evaluates the quality of the codebase in terms of its structure, readability, and reusability. Well-organized code with proper naming conventions, comments, and modular design ensures that the project is maintainable and can be extended or reused in future development.

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1	Consistent Indentation	Uniform spacing/tabs used through-Out the code	Yes	Standard PEP 8 4-space indentation enforced strictly across all Python scripts (app.py, training scripts).
2	Proper File Structure	Files and folders are logically organized	Yes	Dedicated root directories split logically into /static (CSS), /templates (HTML frontends), and root (ML scripts).
3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	Standard programmatic data references used explicitly throughout (e.g., life_expectancy, gni_per_capita, predicted_hdi).
4	Function / Method Names	Functions are descriptively named	Yes	Named modular operations accurately map behaviors cleanly (e.g., clean_missing_data(), predict_wellbeing_index()).
5	Code Comments	Inline and block comments explain logic	Yes	Inline explanations clarify backend routing loops and tracking checkpoints within Flask computational endpoints.
6	Modular Design	Code is split into reusable functions/modules	Yes	Analytical computation logic completely decoupled from frontend web serving routines to maximize performance.
7	No Redundant Code	Duplicate or unused code is removed	Yes	Refactored predictive matrices to load coefficients statically without redundant re-training passes.

8	Error Handling	Exceptions and errors are handled gracefully	Yes	Explicit try-except blocks trap bad form submissions or out-of-bounds metrics to return friendly user alerts.

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level (High / Medium / Low)
1	Data Cleaning Pipeline	Python / Pandas	Initial training preprocessing phase & runtime backend sanitization layer.	High
2	Model Predictor Interface	Python / Scikit-Learn	Local testing validation scripts & Flask main routing execution node.	High
3	Response Form Field Template	HTML5 /Bootstrap	User data simulation portal & future development administrative update dashboards.	High
4	Input Validation Matrix	JavaScript / Python	Client-side frontend typing locks & server-side range boundary validation checks.	High

Overall Code Quality Assessment:

Aspect	Rating (1-5)	Comments
Code Layout & Structure	5	Clean folder schema segregates static resources from predictive application layers seamlessly.
Readability	5	Strict PEP 8 standard code styles and highly intuitive naming rules remove structural cognitive friction.
Reusability	4	Decoupled prediction script components make it simple to slot in newer model variants later on.
Documentation / Comments	4	Extensive block documentation explains the structural endpoints and model serialization parsing methods.
Overall Score	4.5	Highly maintainable and clear deployment codebase architecture.